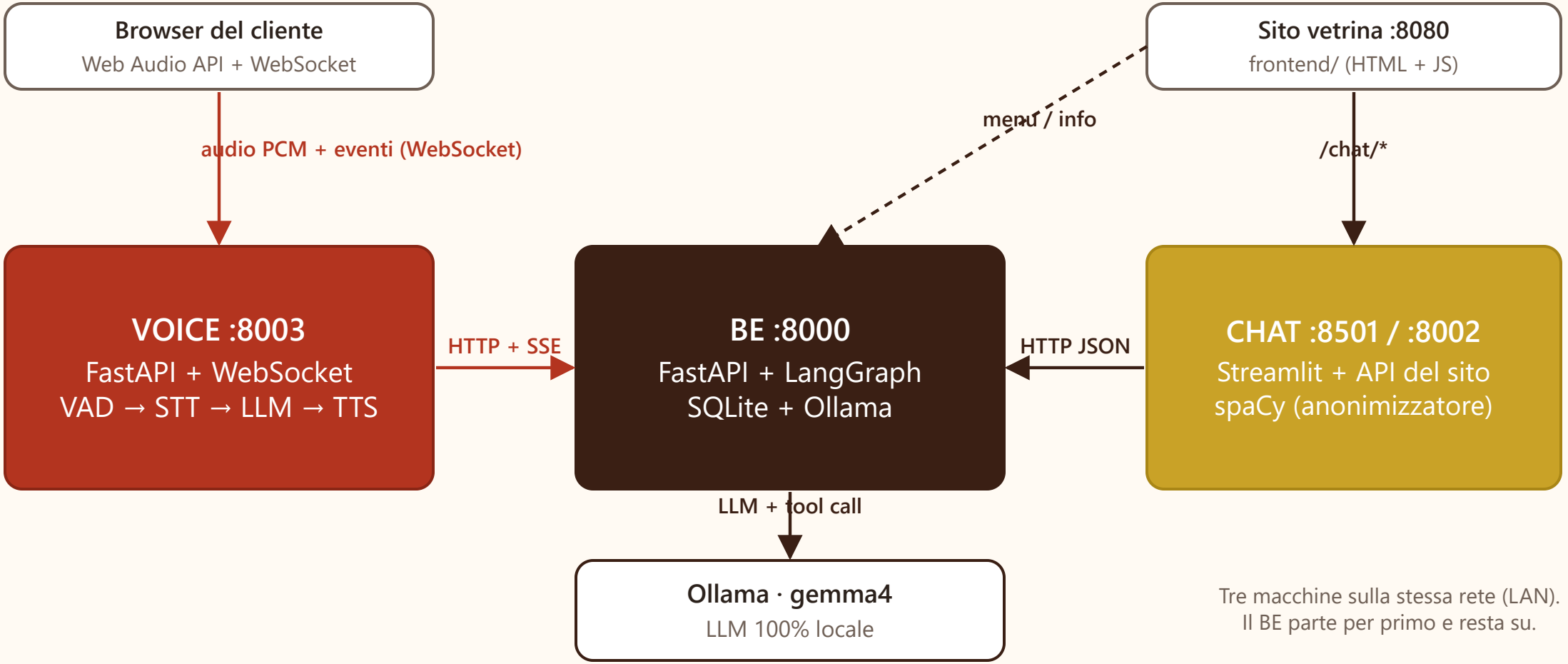
PRESENTAZIONE TECNICA

Stack tecnologico

Ambiente (venv), rete (LAN) e componente Voice: cosa usiamo, come funziona e perché l'abbiamo scelto

Il sistema in una pagina



Tre macchine sulla stessa rete (LAN).
Il BE parte per primo e resta su.

Ambiente: un venv condiviso, requisiti per ruolo

Un solo `.venv` alla radice del repo

- **base.txt** (sempre): fastapi, uvicorn[standard], httpx, pydantic, pydantic-settings, python-dotenv
- **be.txt**: langchain, langgraph, langgraph-checkpoint-sqlite, langchain-ollama, sqlalchemy
- **chat.txt**: streamlit, requests, spacy, phonenumbers
- **voice.txt**: websockets, numpy, faster-whisper, piper-tts, onnxruntime, sounddevice, phonenumbers, webrtcvad-wheels *(opzionale)*
- **dev.txt**: pytest, pytest-asyncio, mypy

Come resta coerente

- Ogni macchina installa **base + il suo ruolo** (venv leggero)
- `.venv/.setup_state.json` salva lo **sha256** di ogni file requirements
- Se un collega cambia un file, al prossimo avvio `is_setup()` vede l'hash diverso e riparte da solo
- `--role all` = tutto, per sviluppo e test
- Pacchetti che richiedono compilatore C++ sono **opzionali**: se falliscono si va avanti con un avviso

Python ≥ 3.10 (consigliato 3.11/3.12; qui 3.13.5). Il setup usa **solo la libreria standard**: funziona anche con un venv rotto.

setup/setup_env.py: cosa fa, in ordine

1 Controllo Python
≥ 3.10

2 Crea .venv
se manca

3 pip install
pacchetto per pacchetto, output live

4 Editable install
pip install -e voice

5 Modelli
spaCy · Piper · Ollama

6 Scrive lo stato
hash dei requirements

Se un pacchetto richiesto fallisce
il setup si ferma con il nome del pacchetto e riparte da lì: i passi già fatti si saltano

Se un pacchetto OPZIONALE fallisce
es. webrtcvad senza compilatore C++ → avviso e si continua: il VAD a energia è il default

Download dei modelli: voce Piper italiana da Hugging Face (~28 MB) in voice/models/ · modello spaCy it_core_news_md (solo CHAT) · Ollama: ollama pull del modello (solo BE)

entrypoint/avvia.py: un comando per macchina

1 git pull --ff-only

2 Identità: ruolo + IP

3 Setup venv (per ruolo)

4 .env di macchina

5 Firewall (netsh)

6 Attesa del BE

7 Avvio componente

git: solo fast-forward
mai merge automatici sporchi

lan/macchina.json
ruolo + IP rilevato (socket UDP)

.env generato da .env.example
BACKEND_BASE_URL=http://IP-BE:8000

Firewall
regola TCP in ingresso per ogni porta

Cancello sul BE
polling di /api/health ogni 2 s

Comandi

python entrypoint\avvia.py
--status stato delle 3 macchine
--attendi il presentatore aspetta
--sono identifica e basta
--no-git --no-wait --yes

Avvio automatico
Scheduled Task al login +30 s:
la macchina parte da sola

L'entrypoint nel dettaglio: funzione per funzione

#	Funzione	Cosa fa	Se qualcosa va storto
1	`git_pull()`	`git pull --ff-only`	si ferma e stampa l'errore: **non forzare**
2	`resolve_role()` · `ensure_identity()`	ruolo da `--role` / `LAN_ROLE` / `macchina.json` / domanda; IP rilevato	lo chiede una sola volta
3	`ensure_environment(role)`	confronta gli hash e installa solo se serve	si ferma: rilanciare dopo aver sistemato
4	`configure_env()`	scrive `BACKEND_BASE_URL`, `BE_URL`, `HOST` nel `.env`	errore solo se manca `.env.example`
5	`firewall_hint()`	`netsh advfirewall` per ogni porta del ruolo	**non blocca**: stampa il comando da admin
6	`wait_for_be()`	`GET /api/health` ogni 2 s, max 180 s (solo CHAT e VOICE)	«BE assente»: il componente non parte
7	`start_component()`	processo principale + servizi extra, log live	`Ctrl+C` ferma tutto

Codice: `entrypoint/avvia.py` (`main`) e `lan/machine_setup.py`. `--status` e `--attendi` usano gli IP di `lan_config.json`: con IP statici si fissano con `--pubblica-ip`.

LAN: ruoli, porte e cancello sul BE

Ruolo	Cartella	Porta	Health	Servizi aggiuntivi
BE	`backend/`	8000	`/api/health`	+ sito vetrina :8080 (http.server su frontend/)
CHAT	`chat/`	8501 (Streamlit)	`/_stcore/health`	+ API del sito :8002 (uvicorn chat/api.py)
VOICE	`voice/`	8003	`/health`	WebSocket /ws/voice · sito / · test /test-voice

Regole di avvio

- Ordine: **BE → CHAT → VOICE** (`ordine_avvio` in lan_config.json)
- CHAT e VOICE **non partono** finché il BE non risponde (gate di 180 s)
- Tutto è dichiarato in `lan/lan_config.json`: comando, cwd, porte, health, variabili

Perché così

- Gli IP in lan_config.json sono **esempi**: quelli veri si chiedono una volta e si ricordano in macchina.json (non in git)
- Così il giorno della demo il presentatore tocca **un solo PC**: `avvia.py --attendi`

Voice: lo stack a strati

Browser / client	Web Audio API (ScriptProcessor, 2048 campioni) · WebSocket · canvas (orb) live_client: microfono e altoparlante con sounddevice
Trasporto	FastAPI 0.143 + uvicorn 0.54 · websockets 16 asyncio.Queue con dimensione massima (backpressure)
Audio locale	VAD a energia RMS (opzionale webrtcvad) · faster-whisper 1.2.1 (CTranslate2): small, int8, CPU Piper 1.8 + onnxruntime 1.31 · voce italiana it_IT-riccardo-x_low
Conversazione	orchestrator · contacts (phonenumbers) · anonymizer (regex + spaCy 3.8) toolguard · prompt in italiano · messages / spoken (numeri e date a parole)
Client del BE	httpx 0.28 asincrono · retry con backoff · SSE (/api/chat/stream) pydantic-settings per la configurazione (.env)
Qualità	pytest 9 + pytest-asyncio · mypy strict (29 file) · log JSON strutturati doctor / diag · Dockerfile multi-stage, utente non-root, healthcheck

Qualità: test, tipi e diagnosi

192
test automatici (voice)

29
file con mypy --strict

12
controlli del doctor

3
ruoli: be · chat · voice

Come proviamo senza il resto del sistema

- **Stub BE** (voicebot/devtools/stub_be.py): un BE finto con registrazione, conferma sì/no, modifica, cancellazione
- Test **ermetici**: conftest isola dal .env e spiega l'interprete sbagliato
- `voice local`: stub + server + browser con un comando

Come capiamo cosa non va

- `python -m voicebot doctor`: python, .env, pacchetti, modelli, audio, porta, BE, registrazione
- `/diag` e tab STATO della pagina di test mostrano lo stesso report
- Log JSON a una riga: facili da filtrare

Decisioni di stack e motivi

Scelta	Alternativa scartata	Perché
Tutto locale (Ollama, Whisper, Piper)	API cloud	Vincolo di progetto: nessun dato personale fuori dalla rete
Un venv, requisiti per ruolo + hash	Un requirements.txt unico	Evita conflitti di merge e installazioni pesanti inutili
Setup e LAN in sola stdlib	Script con dipendenze	Funzionano anche quando il venv è rotto
asyncio con code limitate	Thread / code illimitate	Barge-in e backpressure semplici, niente memoria che cresce
VAD a energia di default	webrtcvad obbligatorio	Zero dipendenze compilate su Windows
Whisper small (non tiny)	tiny (più veloce)	tiny sbaglia numeri e nomi: per prenotare non basta
Pacchetti opzionali tollerati	Setup che si blocca	Un compilatore C++ mancante non deve fermare la demo

Comandi da ricordare

Cosa voglio fare	Comando
Prima volta su una macchina	`python entrypoint\avvia.py`
Solo il setup, per ruolo	`python setup\setup_env.py --role voice --with-dev`
Verifica veloce (esce 0/1)	`python setup\setup_env.py --check-only`
Test completo in locale (stub BE)	`voice\voice local`
Voice contro il BE vero	`voice\voice real → http://localhost:8003/test-voice`
Perché non funziona?	`python -m voicebot doctor`
Test + tipi	`cd voice · python -m pytest tests -q · python -m mypy src`
Stato delle tre macchine	`python entrypoint\avvia.py --status`

PRESENTAZIONE TECNICA

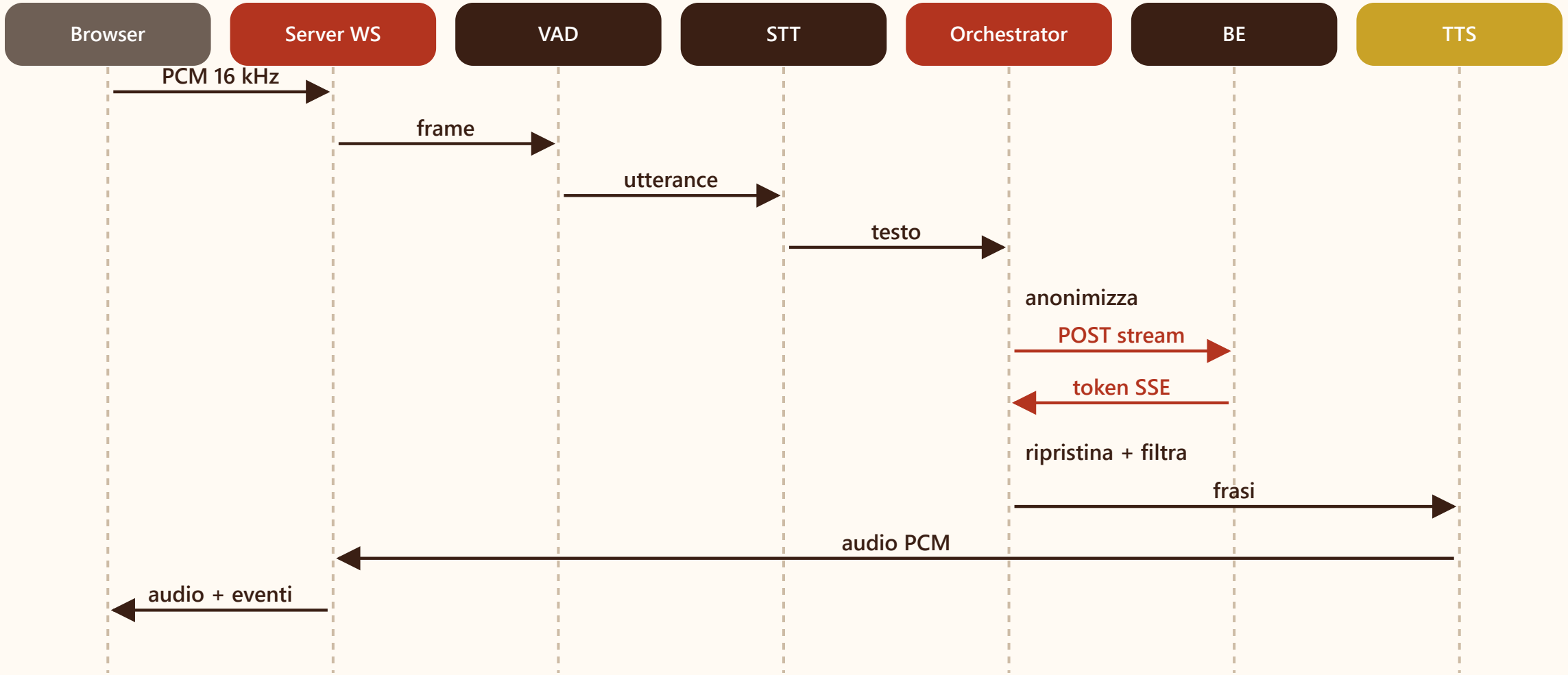
Il flusso del codice Voice

Dal microfono del browser alla risposta parlata: moduli, code, stati, privacy e punti di contatto con il BE

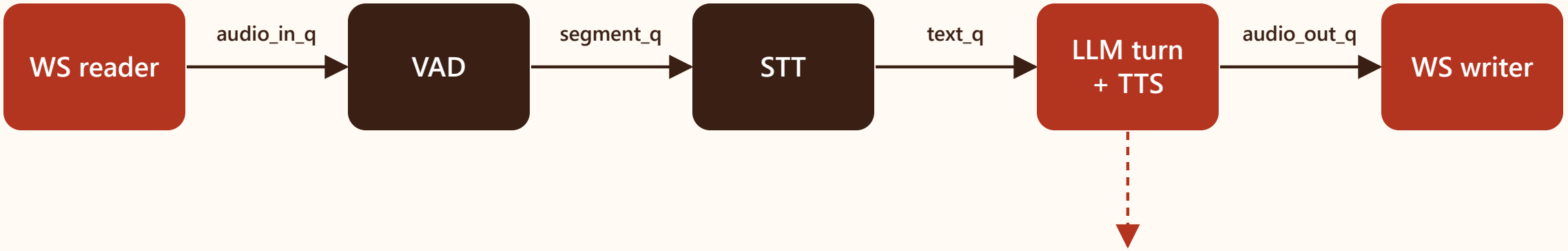
Mappa del codice: 5 livelli

Trasporto	pipeline/server.py (FastAPI, WebSocket, /site/*) · pipeline/session.py (VoiceSession: le 3 fasi) __main__.py (comandi)
Audio	vad.py (energia / webrtc) · stt.py (faster-whisper) tts.py (Piper) · audio.py (PCM, frasi, speakable)
Conversazione	orchestrator.py · contacts.py (registrazione) · anonymizer.py + pii.py · toolguard.py messages.py · prompts.py · spoken.py
Verso il BE	backend.py (httpx, retry, SSE) chat_service.py (alternativa: passa dal servizio CHAT)
Strumenti	doctor.py + diagnostics.py · local.py + devtools/stub_be.py · live_client.py v2.py · config.py · logging_json.py

Una chiamata, dall'orb al BE e ritorno



VoiceSession: tre fasi collegate da code



Ogni coda è LIMITATA (backpressure): se il server è in ritardo si perde il frame più vecchio, mai memoria che cresce

Barge-in (interruzione)

parlato ≥ 300 ms mentre il bot parla → annulla il turno, svuota l'audio in coda, avvisa il client

Contatore di generazione

ogni interruzione incrementa _gen: il lavoro «vecchio» si accorge di essere scaduto e si ferma

VAD: quando una persona ha finito di parlare



Regole

soglia energia 500 · pausa 600 ms · voce minima 250 ms
segmento massimo 15 s (poi taglia comunque)

Il bug che abbiamo risolto

il tempo si misura sull'audio ricevuto, non sul numero di frame: il browser manda 128 ms alla volta, il client Python 20 ms → prima la pausa era 6 volte troppo lunga

Due classificatori, una sola macchina a stati: EnergyVAD (default, zero dipendenze) e WebRtcVADWrapper (opzionale)

Riconoscimento e sintesi: scelte e trucchi

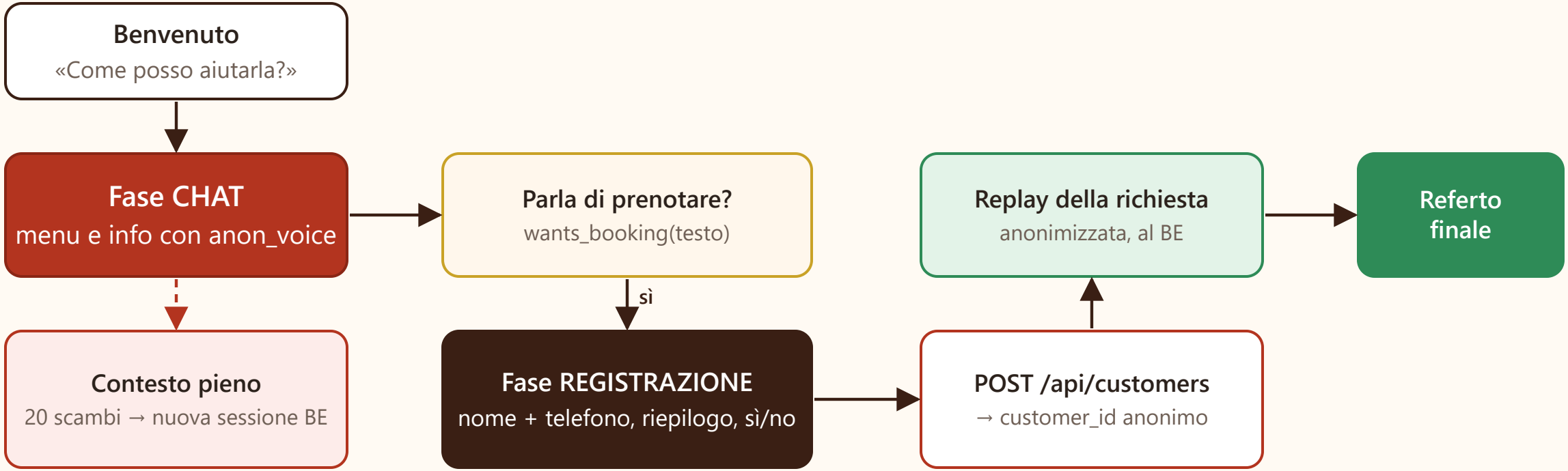
STT · faster-whisper

- Modello **small**, int8, CPU: caricato **una volta** e condiviso da tutte le sessioni
- `beam_size=1`, lingua dall'.env (italiano), `vad_filter=False` (il VAD è già nostro)
- Scarta i segmenti con `no_speech_prob ≥ 0.6` e le «allucinazioni» note di Whisper
- Un **lock**: una trascrizione alla volta; gira in un thread per non bloccare l'event loop
- Prompt iniziale **spento** di default: veniva ripetuto al posto del nome

TTS · Piper

- Voce **it_IT-riccardo-x_low** (onnxruntime), frequenza annunciata al client nell'evento `ready`
- **SentenceChunker**: i token del BE diventano frasi complete e Piper parla **appena la prima è pronta**
- `speakable()`: via markdown ed emoji; i codici diventano «R E S, 9 A 8 B…»
- `spoken.py`: numeri, date e orari a parole («alle venti e trenta»)
- API di Piper ≥ 1.3 gestita (la 1.8 installata rompeva il TTS)

Orchestrator: le fasi della conversazione



Il BE decide tutto il resto (strumenti, conferma sì/no, allergie). Il voice decide solo QUANDO chiedere i dati di contatto.

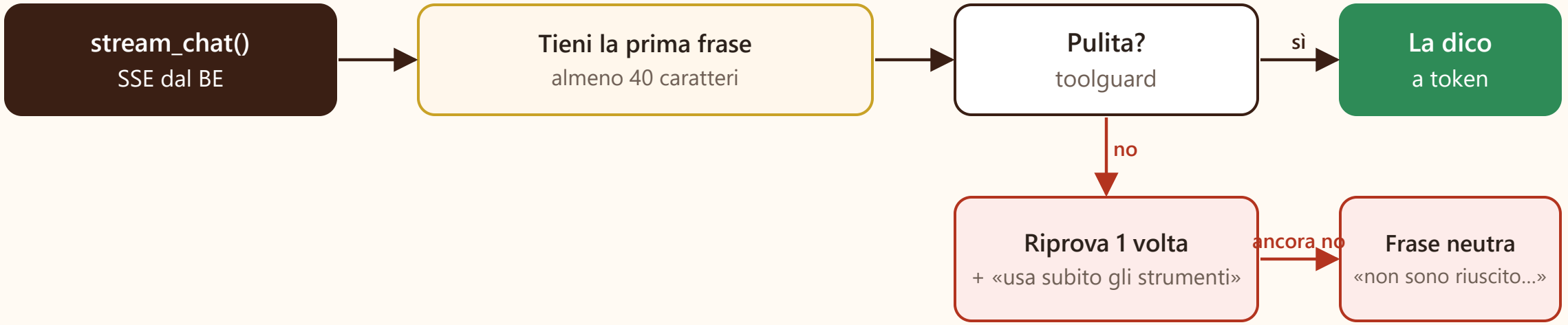
Privacy: dove vanno i dati personali

Dato	Da dove nasce	Dove può andare	Cosa vede l'LLM
Nome e telefono	dette a voce in fase REGISTRAZIONE	**solo** `POST /api/customers` → tabella *customers*	nulla: vede solo `cust_…`
Frasi libere	es. «sono Mario, il mio 333…»	`anonymize()` → `[NOME_1]`, `[TELEFONO_1]` → BE	segnaposto opachi
Risposta del BE	può contenere segnaposto	`restore()` prima della voce (mai mezzo segnaposto detto)	—
Ultima rete	tutto ciò che esce verso il BE	`scrub_pii()`: regex senza stato su e-mail, telefono, CF, indirizzo	—
Log	telefono nei log	mascherato: solo i primi 4 caratteri + «***»	—

Strati dell'anonimizzatore

- 1) regex per forme fisse (e-mail, CF, telefono, indirizzo)
- 2) regex per le frasi che introducono un nome («a nome di…»)
- 3) spaCy `it_core_news_md` come rete di sicurezza (opzionale)

Il turno col BE: il filtro sulle risposte



Cosa intercetta il filtro

nomi di strumenti (get_menu…) · «devo prima chiamare…» · «vuole che verifichi…?» · «Posso mostrarle i piatti…» detto al posto di mostrarli

Referto finale e conferma vocale

Il BE conferma
nel testo c'è RES-XXXXXX

→

GET /api/reservations/{code}
solo il proprietario (altrimenti 404)

→

Referto detto A VOCE
costruito dal codice, non dall'LLM

«Sì» breve (≤ 6 parole, senza «ma»/numeri)
→ evento reservation_confirmed → il popup si chiude dopo 4 s

«Sì, ma alle 21»
NON è una conferma: va al BE come richiesta di modifica

«No»
«Mi dica cosa vuole cambiare»

Eventi verso la pagina
reservation (nome, data, ora, persone, ambiente se chiesto)
registered · reservation_confirmed · call_ended

Contesto, cache e prestazioni

PROBLEMA

Il BE tiene tutto il dialogo come contesto dell'LLM: con un modello locale su CPU più contesto = più lento; il BE inoltre scrive data e `customer_id` in cima al prompt, quindi ogni nuovo id fa rileggere tutto.

COSA FA IL VOICE

- **20 scambi** senza arrivare a una prenotazione → nuova sessione BE + recap parlato
- Al **3° ripristino** avvisa («potrei dover ricominciare»); al **4°** chiude la chiamata (evento `call_ended`)
- Prenotazione raggiunta → contatori a zero; dopo il «sì» finale sessione nuova
- Cliente resta registrato: nessuno ripete nome e telefono

Prestazioni su portatile

- **Warm-up** all'avvio: un turno finto al BE, così il primo cliente non aspetta ~1 minuto
- Id anonimo **condiviso** `anon_voice`: stessa cache del prompt per tutti gli anonimi
- Modello con contesto 8k e 6 thread (`voice/local_llm/Modelfile`)
- Misurato: primo turno ~41–70 s, poi 1–5 s

Protocollo WebSocket /ws/voice

Verso	Contenuto	Note
↑ binario	PCM16 mono 16 kHz	browser: blocchi da 2048 campioni (128 ms) · client Python: 20 ms
↑ testo	{"cmd":"eos"} · {"cmd":"reset"}	fine flusso · «nuova conversazione»
↓ binario	PCM16 al tts_sample_rate (22050)	l'audio del bot, a pezzi, frase per frase
↓ eventi	loading · ready · heard · transcript · thinking · reply · barge_in	stati della chiamata per l'interfaccia
↓ eventi	registered · reservation · reservation_confirmed · call_ended · error	fatti strutturati: popup, «Le mie prenotazioni», fine chiamata

Indirizzo: `ws://HOST:8003/ws/voice` · timeout di inattività 60 s · un solo socket per chiamata.

Il sito e l'interfaccia

Pagine servite dal server voice

- `/` il sito: menu, info, «Le mie prenotazioni», orb, chat
- `/test-voice` pagina di test con tab TECNICA / STATO, trascrizione ed eventi
- `/site/menu · info · reservation/{codice}`: il browser non parla mai col BE, passa da qui
- `/site/chat/*` → servizio CHAT (spento qui)
- `/health · /ready · /diag`

L'orb parla da solo

- **Pallini bianchi** che vanno al centro = il BE sta pensando
- **Anello dorato** sottile e lento = a riposo
- Anello che **si ispessisce** con la fascia chiara che cresce = sta per parlare
- Anello che **ondeggia** (non in sincrono con l'orb) = sta parlando
- **Popup** di riepilogo: si chiude 4 s dopo il «sì» finale, solo a voce

Resilienza: cosa succede quando qualcosa va storto

Situazione	Cosa facciamo	Perché
BE irraggiungibile	retry con backoff esponenziale	se fallisce: frase parlata «non riesco a raggiungere il server»
Timeout in lettura su POST	**mai ritentato**	la richiesta potrebbe essere già stata eseguita: niente doppioni
Stream interrotto	riconnessione solo prima del primo token	dopo, si chiude: riprovare ripeterebbe parole
Rete del client	live_client / pagina: riconnessione con backoff	la chiamata sopravvive a una caduta
Modelli non pronti	evento `loading`, attesa fino a 120 s	il caricamento parte all'avvio (PRELOAD_MODELS)
Errori tecnici	mai letti ad alta voce	sempre una frase breve e gentile

PRESENTAZIONE TECNICA

Storia del progetto: problemi ricorrenti e come li abbiamo risolti

Ricostruita dalla cronologia git: 90 commit in tre giorni, tre contributori, tre componenti

Il progetto in numeri

90
commit

3
contributori

3
giorni (14–16 ottobre)

192
test del solo voice

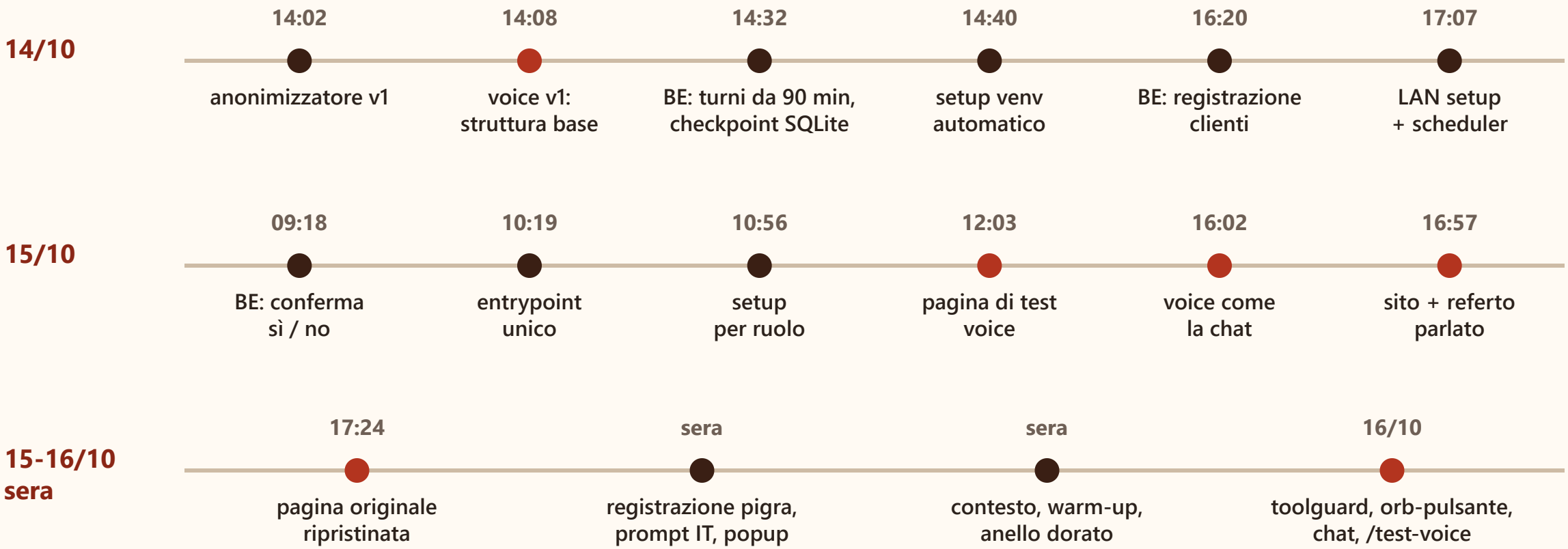
Dimensioni (righe)

Parte	Righe
voice (codice, pagine, sito)	6.272
voice (test)	2.358
backend	2.809
chat	1.420
frontend	1.212
lan + setup + entrypoint	1.327
test del backend	1.738

Ritmo

- 14/10: **39** commit — nascono i tre componenti
- 15/10: **50** commit — setup per ruolo, entrypoint, sito, rifacimento del voice
- 16/10: rifiniture (orb, popup, contesto)
- ~21.000 righe aggiunte, ~4.000 tolte nel periodo

Linea del tempo



rosso = lavoro sul componente VOICE · marrone = BE, CHAT, setup e LAN

Ricorrente 1 · Setup del venv e conflitti di merge

PROBLEMA

Un solo `requirements.txt` e `setup_env.py` toccati da più persone: **7 modifiche su 2 giorni**, commit come «mio setup: tenere questa versione» e «fix setup env dopo merge sbagliato»; installazioni pesanti su macchine che non servivano.

COME L'ABBIAMO RISOLTO

- Cartella spostata e stabilizzata: `tools/env_setup` → `setup/`
- **Requisiti per ruolo** (base, be, chat, voice, dev): un file per persona → niente conflitti
- Stato in `.venv/.setup_state.json` con **hash sha256**: se un file cambia, il setup riparte da solo
- Installazione **pacchetto per pacchetto** con output live e pacchetti opzionali tollerati
- Guardia: il venv non può mai nascere fuori dalla repo

Catena: b8c1712 (primo setup) → 1aa2df6 → 1c172bd → 58e1f38 → 5af2bf3 (fix dopo merge) → 3fc4dbb (per ruolo)

Ricorrente 2 · Ogni macchina deve sapere chi è e dove sta il BE

PROBLEMA

Tre PC, tre ruoli, IP che cambiano, avvio nell'ordine giusto, firewall: 7 modifiche a `lan/machine_setup.py` in due giorni («fix su lan per dare i ruoli a ciascuna macchina»).

COME L'ABBIAMO RISOLTO

- **Identità** della macchina in `lan/macchina.json` (ruolo + IP rilevato con un socket UDP, mai in git)
- **Un solo comando**: `entrypoint/avvia.py` (git → identità → setup → .env → firewall → attesa BE → avvio)
- **Cancello sul BE**: CHAT e VOICE aspettano che `/api/health` risponda
- **Servizi extra** per ruolo in `lan_config.json` (API del sito :8002, frontend :8080) fermati insieme con Ctrl+C
- **Avvio automatico** con Scheduled Task per la demo: il presentatore tocca un solo PC (`--attendi`)

5c0ad2b (ruoli) → 96301d1 (entrypoint) → 3fc4dbb (identità) → d183fb2 → 81084e4 (servizi extra)

Ricorrente 3 · La pipeline vocale col modello vero (cd1be40)

Sintomo	Causa	Fix
TTS si rompeva	Piper 1.8 ha cambiato API	gestite sia le API ≤ 1.2 sia ≥ 1.3
Voce con tono sbagliato	frequenza del modello ignorata dal client	annunciata nell'evento `ready`
Prima chiamata lentissima	STT/TTS ricaricati a ogni chiamata	modelli condivisi e **precaricati** all'avvio
Pausa 6 volte troppo lunga	VAD contava i frame (il browser ne manda da 128 ms)	tempo misurato sull'audio ricevuto
Bot che non si lascia interrompere	barge-in non annullava un turno in «thinking»	contatore di generazione + rumore breve ignorato
Prenotazioni doppie	POST ritentato dopo un timeout in lettura	**mai** ritentare una richiesta non idempotente
Test che dipendevano dal PC	.env e interprete della macchina	conftest isola, stub BE, 119 test ermetici

Ricorrente 4 · Il parlato non è testo

PROBLEMA

Whisper **tiny** sbaglia numeri e nomi; il suo «prompt iniziale» veniva ripetuto al posto del nome; «alle 20 e 30» diventava 20:00; una pausa dentro un numero di telefono spezzava la frase in due.

COME L'ABBIAMO RISOLTO

- Modello **small** di default; il prompt è **spento** (f073570)
- Cifre parlate normalizzate («tre tre tre…») e **frammenti uniti** tra una frase e l'altra
- Telefono letto **cifra per cifra** e validato con `phonenumbers`
- Il BE finto capisce «alle 20 e 30» (d18d33c, trovato in una prova col modello reale)
- Numeri, date e orari **a parole** prima della sintesi; allucinazioni di Whisper scartate

Ricorrente 5 · I modelli piccoli sbagliano: l'LLM propone, il codice decide

PROBLEMA

llama3.2 chiamava `create_reservation` su saluti e frasi senza senso: prenotazioni fantasma; passava la stringa «null»; inventava piatti per le allergie. Oggi, con gemma4, annuncia gli strumenti invece di usarli.

COME L'ABBIAMO RISOLTO

- **Conferma esplicita** (06d45e1): l'LLM può solo *proporre*; riepilogo scritto dal codice e azione solo dopo un «sì» intero («sì ma alle 21» non vale)
- Argomenti «null»/«nessuna» trattati come assenti; prenotazione proposta solo se nei messaggi c'è una richiesta (3baf68c)
- **Allergie**: risposta costruita dal codice dal menu, mai dal modello (c7c1599)
- **Voice**: filtro `toolguard` sulla prima frase, un nuovo tentativo, poi frase neutra
- Modello scelto automaticamente tra quelli installati che sanno usare gli strumenti (4c7f0a2)

Ricorrente 6 · La privacy decide l'architettura

PROBLEMA
Nome e telefono servono al ristorante ma l'LLM non deve vederli; un nome usato come identificativo era già un dato personale.

COME L'ABBIAMO RISOLTO

- `POST /api/customers` è l'**unico** punto con dati personali → `customer_id` anonimo (6f64a50)
- Con la registrazione attiva solo il **proprietario** legge, modifica o cancella (404 per gli altri)
- Nei tool il `customer_id` arriva **dalla sessione**, mai dal modello
- Anonimizzatore della CHAT portato nel voice con gli **stessi segnaposto**; rete di sicurezza `scrub_pii`
- Registrazione a voce **solo quando serve** (si parla di prenotare); retention: `maintenance purge` dopo 90 giorni

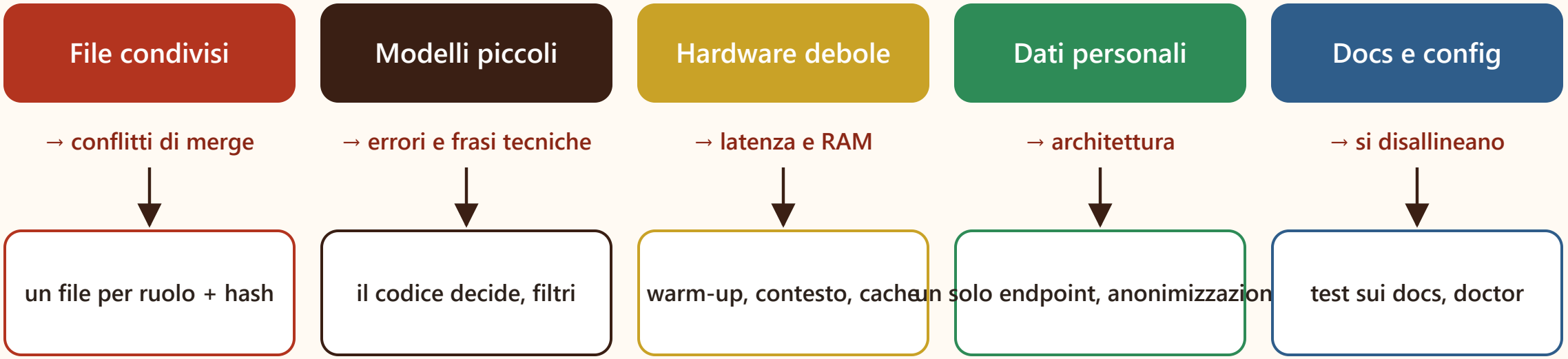
Ricorrente 7 · Docs e configurazione che si disallineano

Cosa	Cosa è successo	Decisione
.env.example	10 modifiche	test_docs controlla che ogni variabile documentata esista davvero
README_VOICE.md	10 modifiche	test che legge il README e verifica moduli e comandi citati
Pagina di test	riscritta e perdute le animazioni	ripristinata dall'originale (55069b6): **estendere, non riscrivere**
Comandi nei docs	`voice\voice` diventato «voiceoice»	backslash mangiati da caratteri di controllo: ripristinati
Branch	lavoro rimasto su savov e non su main	merge di recupero (27b67cc)

Gli ultimi giorni: problema → decisione

Problema	Decisione
Saluto troppo lungo e «avido»	Saluto breve; nome e telefono solo quando si parla di prenotare, poi la richiesta riparte da sola
Bot che rispondeva in inglese per via del prompt	Prompt **in italiano**, lingua del cliente se la conosce
Il cliente non vedeva cosa era stato prenotato	Popup di riepilogo, chiuso 4 s dopo il «sì» finale (solo voce)
Modello locale lento su portatile	Contesto 8k, warm-up, id anonimo condiviso, 20 scambi poi nuova sessione, 4° giro chiude
«Chiamo get_menu… vuole che verifichi?»	Filtro sulla prima frase + un nuovo tentativo
Orb senza segnali chiari	Anello dorato: sottile/lento → si ispessisce → ondeggia mentre parla
Pulsante + pannello con bottoni	Il pulsante «Prenota a voce» diventa l'orb; chat «Scrivi» a parte; test su /test-voice

Le cinque cause dietro i problemi ricorrenti



Sette problemi, cinque cause: ogni decisione di progetto nasce da un incidente reale.

Cosa resta aperto

Rischi e limiti

- La prova **a tre macchine** non è ancora stata fatta dal vivo
- Il BE scrive data e customer_id in cima al prompt: ogni nuovo cliente fa rileggere tutto al modello
- Il BE a volte risponde con testo invece di usare gli strumenti
- Il riconoscimento è fissato sulla lingua dell'.env: altre lingue non sono trascritte bene
- Nessun endpoint «elenco prenotazioni del cliente»: la pagina ricorda i codici nel browser

Prossimi passi

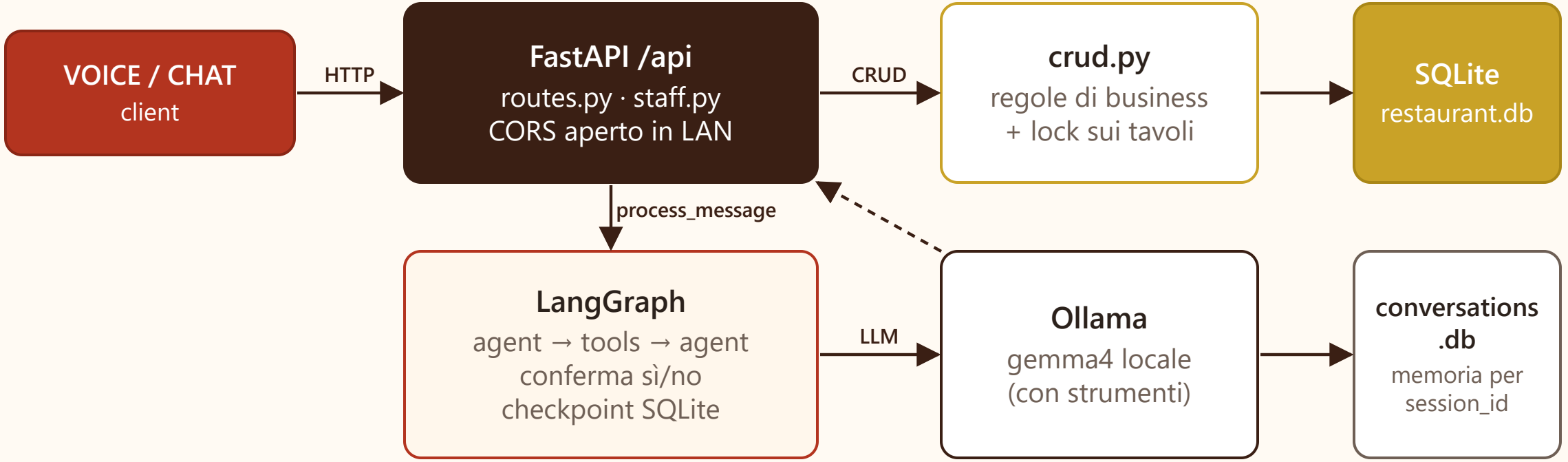
- Spostare le righe per cliente in fondo al prompt del BE
- Allineare il prompt vocale del BE a `voice/src/voicebot/prompts.py`
- Provare il flusso su macchine più potenti e misurare i tempi
- Valutare il cognome solo quando c'è una prenotazione simile (serve il BE)
- Provare Docker (`voice/Dockerfile` non è mai stato costruito qui)

PRESENTAZIONE TECNICA

Il BE e dove il Voice lo incontra

Cosa fa il backend, quali endpoint usiamo, cosa succede dentro a ogni richiesta e cosa chiedere al team BE

Il BE in una pagina



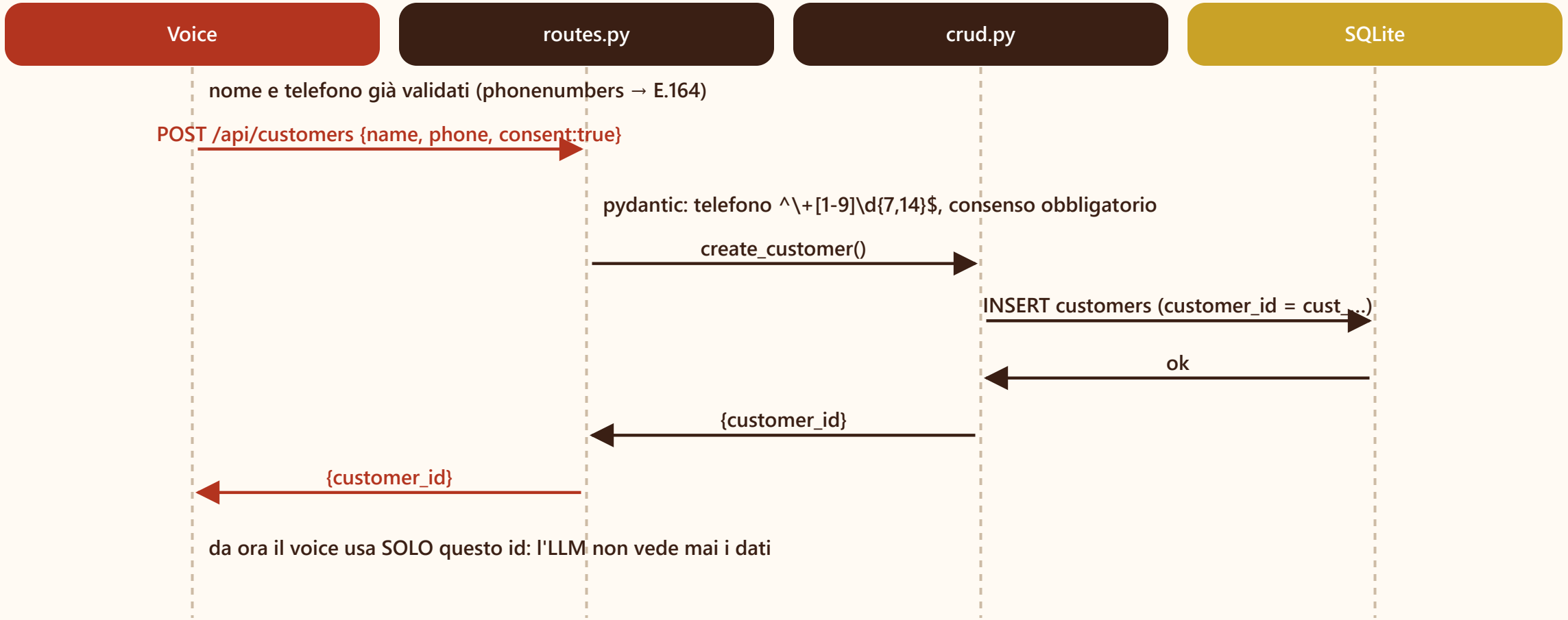
Il BE è l'unico che parla con l'LLM e con il database: i client sono «sottili».

Dove il Voice tocca il BE

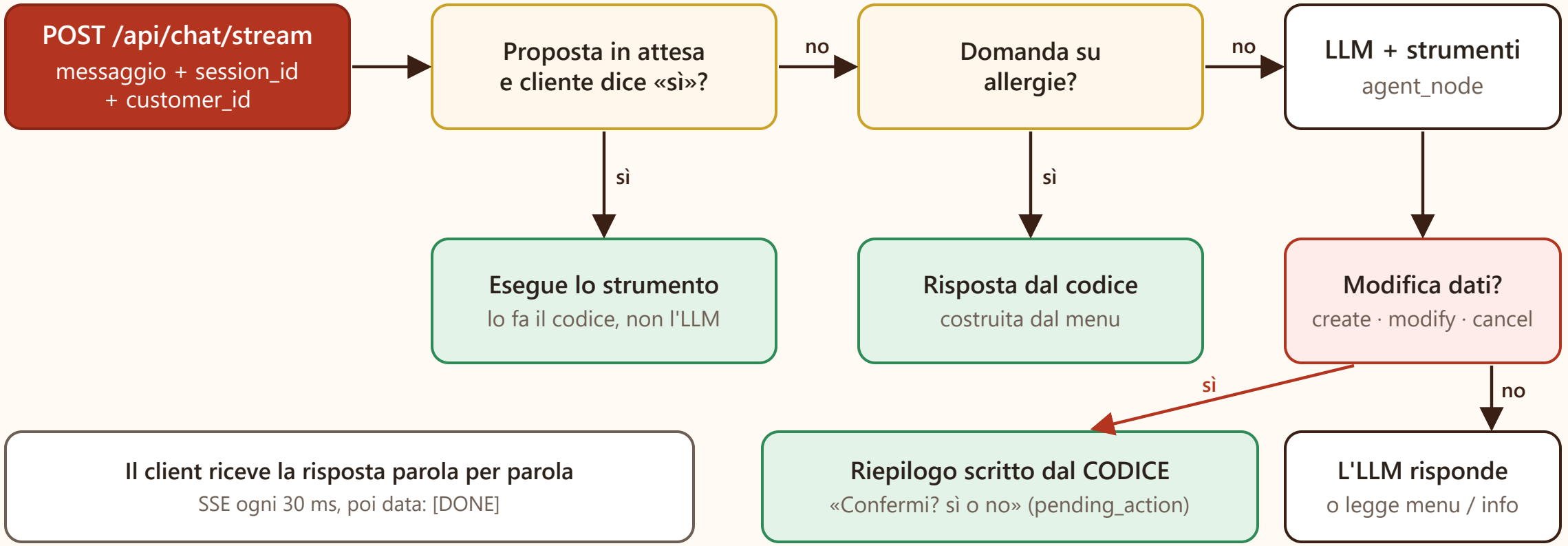
Nel codice voice	Endpoint del BE	Cosa succede
`BackendClient.health_info()`	GET `/api/health` (poi `/health`)	database e modello rispondono davvero: online / degraded
`register_customer()`	POST `/api/customers`	salva nome+telefono, restituisce `cust_…` (unico punto con dati personali)
`chat()`	POST `/api/chat`	un turno intero (usato da v2 e dai referti)
`stream_chat()`	POST `/api/chat/stream`	stessa cosa ma a token SSE; il BE **prima** esegue tutto il turno, poi manda le parole
`get_reservation()`	GET `/api/reservations/{codice}?customer_id=`	solo il proprietario: gli altri ricevono 404
server voice `/site/info`	GET `/api/info`	indirizzo, telefono, orari per il sito
server voice `/site/menu`	GET `/api/menu`	menu con filtri (vegano, senza glutine…)

Tutto passa da `voice/src/voicebot/backend.py` (httpx, retry, timeout) · il browser non parla mai direttamente col BE.

Registrazione del cliente: l'unica volta con dati personali



Dentro un turno di chat (backend/graph/workflow.py)



Il protocollo di conferma: l'LLM propone, il codice esegue

Esempio

```
Cliente  vorrei un tavolo per due domani alle 20
LLM      → propone create_reservation(...)
CODICE   «Riepilogo: tavolo per 2 persone sabato 17
          ottobre alle 20. Confermi? sì o no»
Cliente  sì
CODICE   esegue lo strumento → codice RES-9A8B2C

Cliente  sì, ma alle 21     ← NON è una conferma:
         il BE torna all'LLM come modifica
```

Perché è fatto così

- Un modello debole creava prenotazioni **fantasma** su saluti e frasi senza senso
- Una richiesta impossibile (giorno di chiusura, orario fuori turno, gruppo troppo grande) **non viene nemmeno proposta**
- `is_confirmation()`: vale solo se il messaggio è fatto **solo** di parole di conferma (≤ 4)
- Il voice fa lo stesso a valle: il suo referto finale e il suo «sì» sono **un secondo livello** di conferma, a voce

Dati e regole di business

tables	reservations	customers	menu_items	closures · info
table_number	booking_code (RES-…)	customer_id (cust_…)	category · name · price	closures: data + motivo
capacity (posti)	customer_id	name	allergens	restaurant_info:
location: 3 ambienti	table_id	phone (E.164)	flag: veg · vegan · GF	chiave → valore
is_active	date · time · shift	consent_at	available (esaurito)	
	guests_count			
	status prenotazione			

Regole in crud.py

turno di 90 minuti: un tavolo è occupato se le finestre [ora, ora+90] si sovrappongono · tavolo più piccolo che basta (best fit)
pranzo 12:00–15:00 (ultima 14:00), cena 19:00–23:30 (ultima 22:00) · chiuso il lunedì · lock sull'allocazione dei tavoli

L'LLM nel BE: scelta e velocità

Scelta del modello (llm_provider.py)

- `OLLAMA_MODEL` può essere un elenco in ordine di preferenza + `OLLAMA_FALLBACK_MODELS`
- Si usa il **primo installato che sa usare gli strumenti** (`/api/show` → capabilities): senza strumenti non si prenota
- Nessun modello adatto → 503 con un messaggio chiaro, non un errore tecnico
- La scelta è nel log e in `/api/health`

Velocità e robustezza

- **Thinking spento** di default (3–6 volte più veloce): `OLLAMA_REASONING`
- **Warm-up** all'avvio in un thread: il modello è già in memoria al primo cliente
- `OLLAMA_KEEP_ALIVE=30m`, timeout 120 s, temperatura 0,1
- `python -m backend.check_model`: 8 prove con OK/KO e tempi
- `health` = «online» solo se database E modello rispondono

Il prompt del BE e la cache dell'LLM

Regole fisse e lunghe
ruolo, stile, menu solo da strumento, divieti

«Cliente NON registrato»
blocco che cambia con la registrazione (righe 20–22)

Data odierna
cambia ogni giorno (riga 43)

customer_id
cambia per cliente (righe 48–50, 72)

Istruzioni degli strumenti
usano il customer_id

Misurato su un portatile
primo turno di un cliente: 41–75 s · turni successivi: 1–5 s
il modello legge ~36 token/s, scrive ~7 token/s

Proposta per il BE
mettere data, stato di registrazione e customer_id
in FONDO al prompt: la parte lunga resta in cache

Cosa fa il Voice nel frattempo
id anonimo condiviso anon_voice · warm-up all'avvio ·
contesto limitato · nuova sessione ogni 20 scambi

Oggi: appena cambia una riga rossa, l'LLM rilegge tutto il resto (cache persa).

Cosa chiedere al team BE

Priorità	Richiesta	Perché
Alta	L'endpoint `/api/chat/stream` è `async` ma chiama `process_message` in modo bloccante	mentre l'LLM lavora il BE non risponde a nulla, nemmeno a `/api/health` (osservato: timeout di 10–20 s)
Alta	Spostare data / registrazione / customer_id in fondo al prompt	cache dell'LLM intatta, primo turno per cliente da ~60 s a pochi secondi
Alta	Il modello a volte scrive «Chiamo get_menu…» invece di chiamare lo strumento	il voice lo filtra, ma la causa è nel prompt/modello del BE
Media	Allineare il prompt vocale a `voice/src/voicebot/prompts.py`	oggi il BE legge solo l'ultimo messaggio e usa il suo prompt
Media	Impostare `num_ctx` esplicito per il modello	evita contesti da 64k su macchine con poca RAM
Media	Cognome solo se c'è già una prenotazione simile	serve vedere le prenotazioni di altri: solo il BE può farlo, nel rispetto della privacy
Bassa	Endpoint «prenotazioni del cliente» (`/api/customers/{id}/reservations`)	la pagina potrebbe elencarle tutte, non solo quelle di questo browser